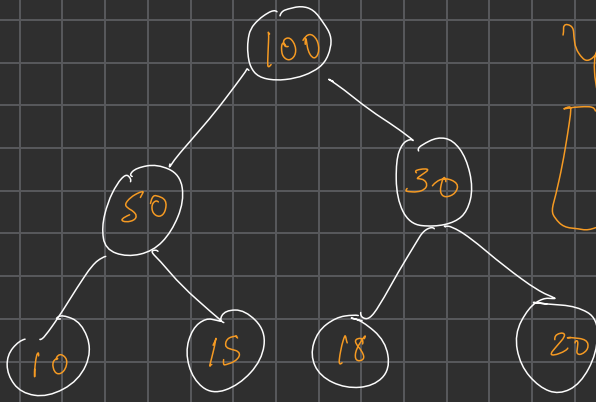
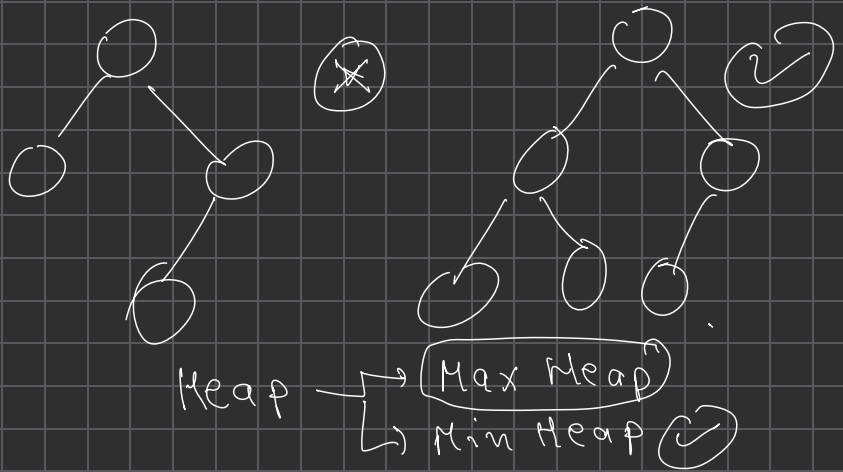




Heap : K.C ✓

Complete Binary Tree

last level should
be filled from left
to right



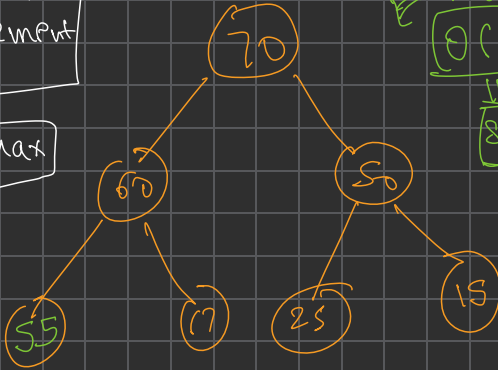
Max heap $\Rightarrow$ CBT
Every parent should be greater than equal to its child

Complete Binary Tree == Height
balance Binary
tree

Max element
root element
hoga

Second Max

60 ✓



Max
 $O(1)$

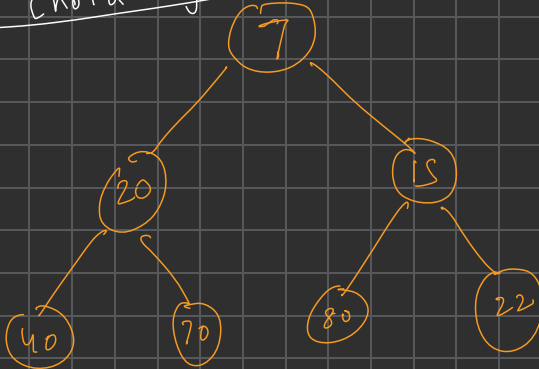
Search

Max elem

Max heap

× Bilkul nahi

Third max
root ke 2 child
Jo chota hoga, wo mer third max

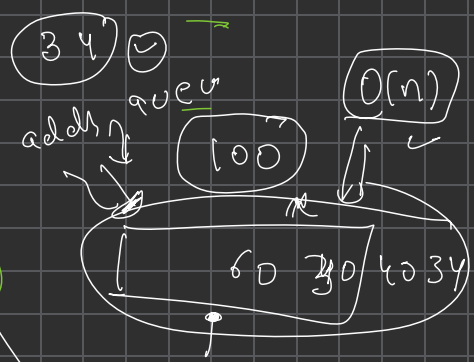
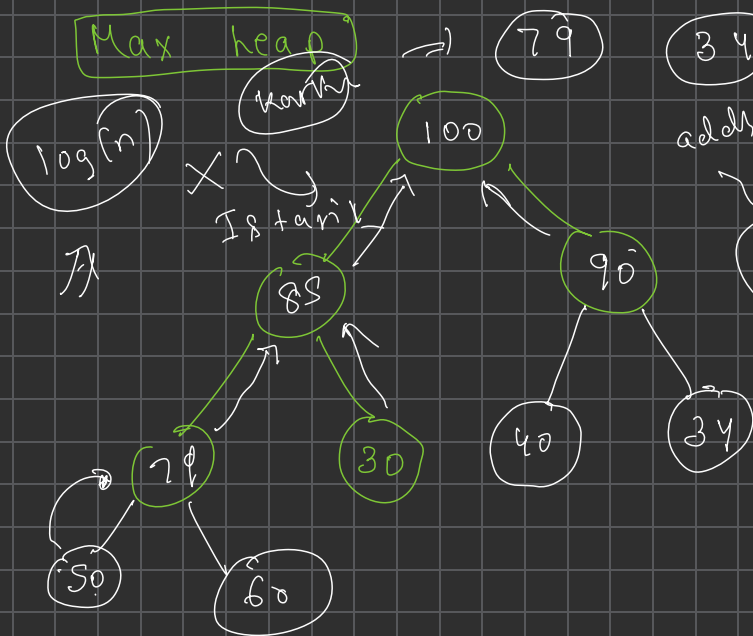


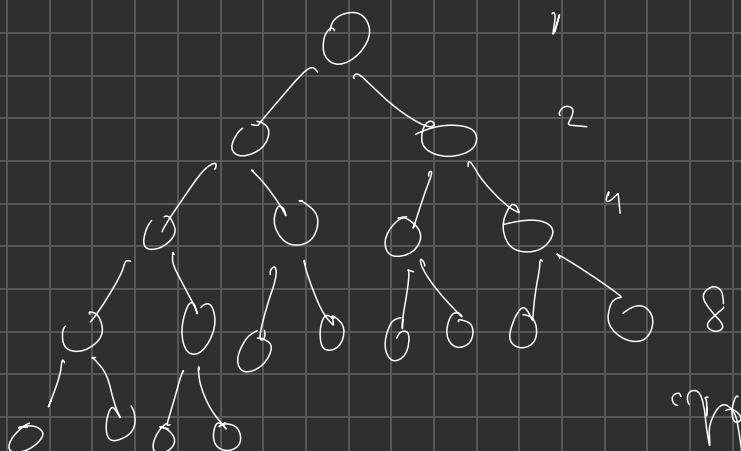
Min-Heap

⇒ Parent should
be less than
equal to its child

✓

Max heap

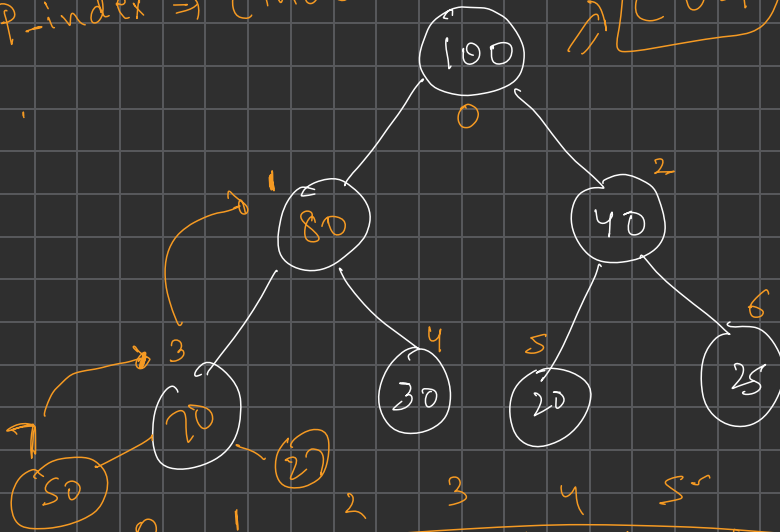




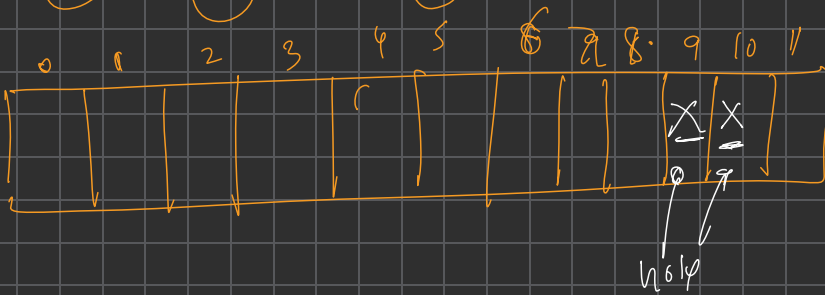
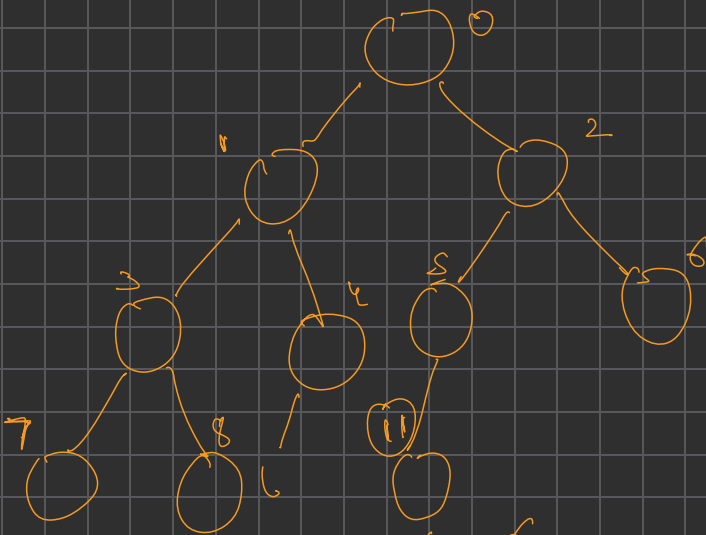
Child = index;
 $P_index \Rightarrow (index-1)/2$

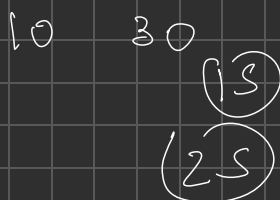
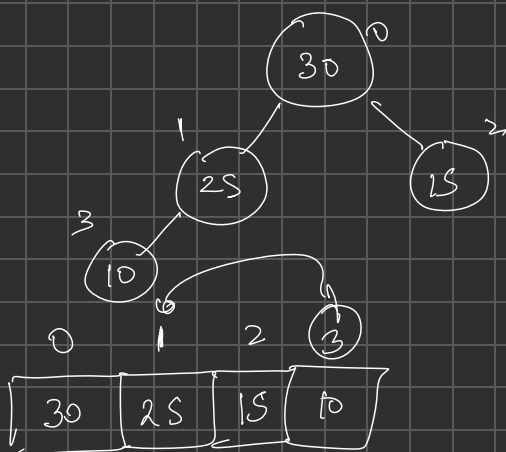
CBT

Complete PM
 Parent = Index
 Leftchild $\Rightarrow 2 * index + 1$
 Rightchild $\Rightarrow 2 * index + 2$

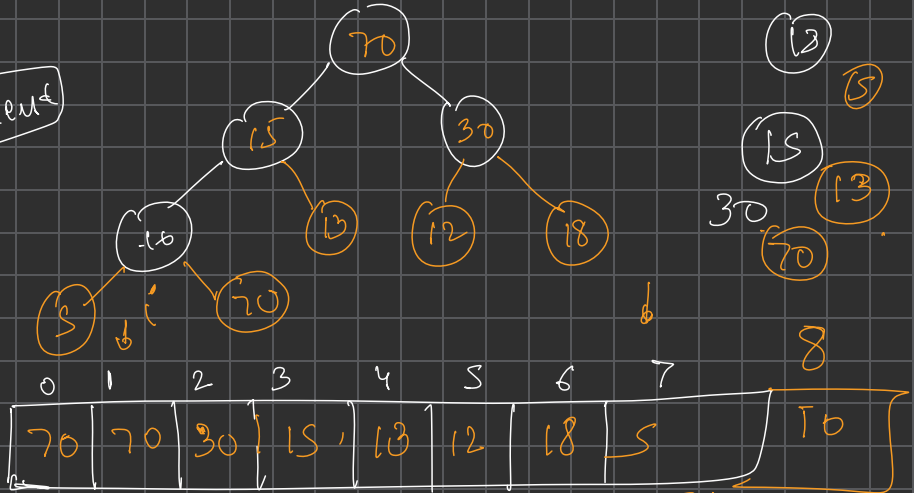


0	1	2	3	4	5	6	7	8
100	80	40	70	30	20	25	50	27





8 element



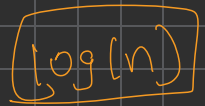
while ($i \geq 0$ & $arr[i] > arr[(i-1)/2]$)

{ swap($arr[i]$, $arr[(i-1)/2]$);

$i = (i-1)/2$

$2 \times \text{index} + 1$

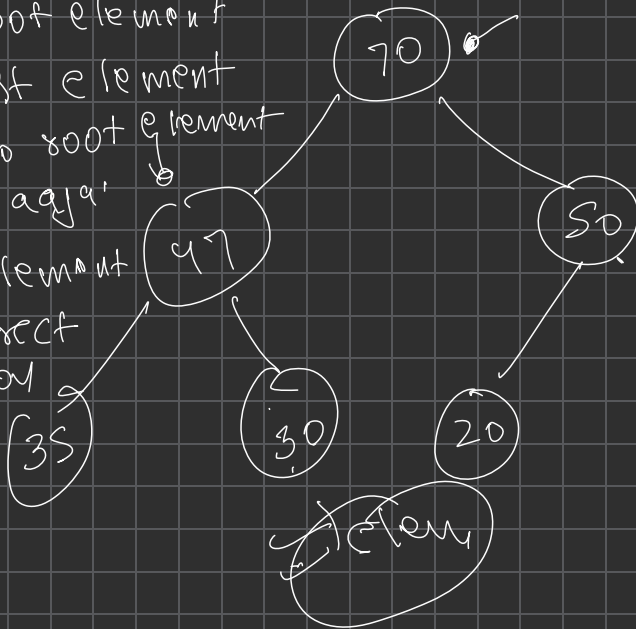
$2 \times \text{index} + 2$



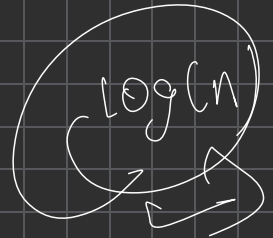
$$n \log n$$

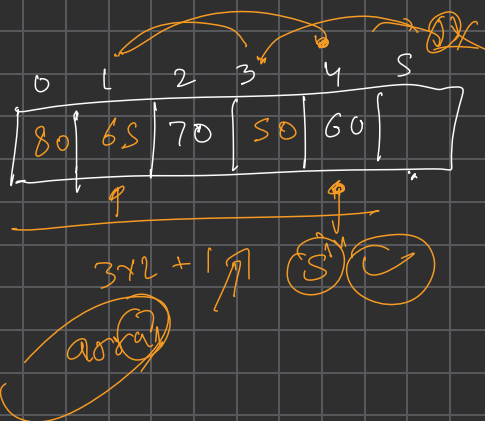
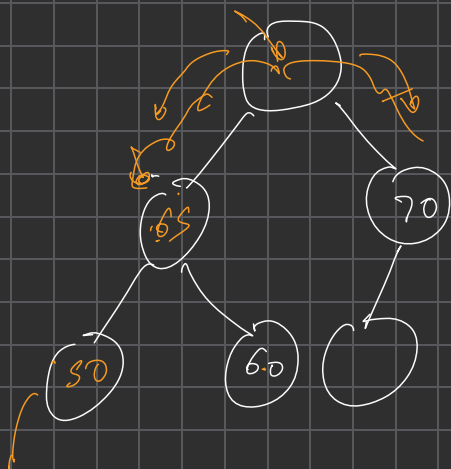
$$n(\log(n) \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} \right))$$

- ① Root element
- ② Last element
or No root element
leke agra
- ③ Or element
in correct
position

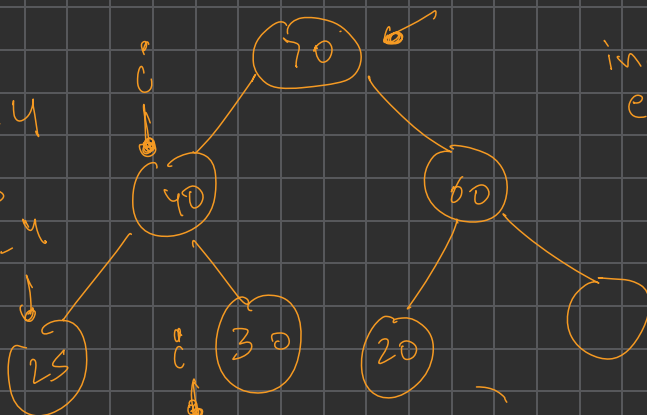


max





target = 4
 Left = 3
 Right = -4



index: next
 element
 where it
 will be
 in

